אפיון מלא (11 חלקים)

עצירה שאי אפשר לזייף. שאי אפשר לנצל נגדך. ושעובדת גם כשהכול סביבה קורס.

חלק 1: חתימה על כל פקודה

רק מי שמוסמך יכול לעצור. כל פקודה עוברת ארבע בדיקות לפני שהשער זז.

איך זה עובד

- המוסמך בונה פקודה: פעולה, תחום, מספר חד-פעמי וחותמת זמן.
- הפקודה נחתמת עם Ed25519. החתימה קטנה, 64 בייט בלבד.
- השער בודק ארבעה דברים: החתימה, הזמן (עד 30 שניות), המספר, וההרשאה.
- רק אם הכול עבר, המצב משתנה.

מה זה מגן מפניו

האיום: תוקף שולח פקודה עם חתימה שלא תואמת.

התשובה: הפקודה נדחית ונרשמת כניסיון חשוד.

הכלים שמאחורי החלק

- **PyNaCl / libsodium (מוכן מהמדף):** ספרייה שיודעת לחתום על הודעה ולבדוק חתימה, כמו חותמת שעווה אישית שאי אפשר לזייף. הגורם המוסמך חותם בה כל פקודת עצירה בחתימת Ed25519, והשער בודק אותה לפני שהוא זז.
- **HashiCorp Vault (מוכן מהמדף):** כספת דיגיטלית לסודות, שמחלקת מפתחות לזמן קצוב ורושמת כל מי שנגע בהם. המפתח הפרטי של החותם שוכן בכספת ומגיע לזיכרון רק ברגע החתימה, ולא יושב כקובץ על הדיסק.
- **HSM / Cloud KMS (מוכן מהמדף):** קופסת ברזל מאובטחת שמחזיקה מפתח וחותמת בתוכה, ורק התוצאה יוצאת החוצה, כמו כספת עם חריץ בלבד. פעולת החתימה מתבצעת בתוך החומרה, כך שהמפתח הפרטי לעולם לא נחשף לשרת ולא לתוקף.
- **mTLS + Envoy (מוכן מהמדף):** שיחה מוצפנת שבה שני הצדדים מציגים תעודת זהות, לא רק השרת, כמו שומר שגם הוא מראה תג בכניסה. Envoy עומד בפתח ה-Gateway ומאמת שגם השולח הוא מי שהוא אומר שהוא, עוד לפני שהפקודה מגיעה ללוגיקה.
- **Open Policy Agent (מוכן מהמדף):** מנוע שמחזיק את חוקי ההרשאות במקום נפרד מהקוד ועונה לכל שאלה בכן או לא. הוא מכריע בבדיקה הרביעית אם המפתח שחתם בכלל מורשה לפעולה ולתחום שביקש.
- **שרשרת ארבע הבדיקות (הרכבנו):** ארבעה שומרים בשורה, ופקודה עוברת רק אם כל אחד מהם נותן אישור, ואם אחד עוצר אותה היא נדחית. כל פקודה נבדקת לפי הסדר: חתימה תקפה, זמן טרי, מספר חד פעמי שלא נוצל, והרשאה לתחום המבוקש.

נתונים

CommandPayload: action (STOP או RESUME), scope (על מי חל), nonce (מספר חד-פעמי), timestamp (מתי נחתם), issuer_key_id (מי חתם)

קלט ופלט: פקודה חתומה ← התקבלה או נדחתה, עם סיבה

חלק 2: אי-חזרה על פקודות

פקודה אמיתית שנשלחה פעם אחת לא תעבוד פעם שנייה.

איך זה עובד

- כל פקודה נושאת מספר אקראי בן 16 בייט.
- השער רושם את המספר בפעולה אטומית אחת (SETNX), בלי חלון להתחכמות.
- אם המספר כבר קיים, הפקודה נדחית כחזרה.
- המספר נשמר 60 שניות, בדיוק כמו חלון הזמן.

מה זה מגן מפניו

האיום: תוקף שומר פקודת RESUME ישנה ושולח אותה שוב כדי לבטל עצירה.

התשובה: המספר כבר רשום, והפקודה נדחית מיד.

הכלים שמאחורי החלק

- **Redis (SETNX + TTL) (מוכן מהמדף):** מאגר נתונים זריז בזיכרון, ופעולת SETNX שומרת ערך רק אם הוא עוד לא קיים, כמו חותמת שנדבקת פעם אחת. השער רושם בו כל מספר חד פעמי שבא עם פקודה, ואם המספר כבר שם, הפקודה נדחית כחזרה על פקודה ישנה.
- **Redis Sentinel / Cluster (מוכן מהמדף):** מערך שמחזיק כמה עותקים של Redis ובוחר מחליף אוטומטי כשאחד נופל, כמו שחקן מחליף על הספסל. הוא דואג שמאגר המספרים החד פעמיים ימשיך לעבוד גם כשאחד מהשרתים מתרסק.
- **Apache Kafka (מוכן מהמדף):** צינור הודעות שמחזיק יומן מסודר של כל מה שנשלח, וכל מי שמאזין קורא באותו סדר בדיוק. פקודת עצירה מתפרסמת אליו פעם אחת, וכל השערים בכל מקום מקבלים אותה ומתעדכנים באותו סדר.
- **Kubernetes NetworkPolicy (מוכן מהמדף):** חוקי תנועה ברמת הרשת שקובעים איזה שירות מותר לו לדבר עם איזה, כמו מעברים סגורים בין חדרים. לצד השער יש רכיב שמאזין לאותו ערוץ ומוריד את המודל לאפס עותקים, עצירה שלא תלויה בקוד השער.
- **Prometheus + Grafana (מוכן מהמדף):** Prometheus אוסף מספרים לאורך זמן, ו-Grafana מציג אותם כמסכי מחוונים חיים, כמו לוח שעונים בקוקפיט. הם מראים כמה ניסיונות חזרה נחסמו, כמה זמן לוקחת הבדיקה, ואם Redis ו-Kafka בריאים.
- **תפוגת המספר = חלון הזמן (החלטה שלנו):** כשהפקודה פגה אחרי 30 שניות, אין טעם לזכור את המספר שלה יותר מכך, כמו כרטיס שכבר אי אפשר להשתמש בו. המספר החד פעמי נשמר 60 שניות, כפול מחלון הסבילות של השעון, ואז נמחק מעצמו.

נתונים

NonceRecord: key (nonce:‹מספר›), value (1 (קיום המפתח הוא המידע)), ttl (60 שניות)

קלט ופלט: המספר מתוך פקודה שעברה אימות חתימה ← חזרה או חדשה

חלק 3: השער

על כל בקשה למודל מחליטים בשבריר שנייה: עוברת או נחסמת.

איך זה עובד

- הבקשה מגיעה עם תחום: איזה מודל, איזה לקוח.
- בודקים את המצב בזיכרון המקומי, בזמן קבוע, בלי פנייה לרשת.
- פעיל: הבקשה עוברת למודל. מושעה: 503 עם הסבר.

- בקשות שכבר התחילו מסיימות (עד 10 שניות). חדשות לא נכנסות.

איזו טעות זה מונע

האיום: עצירה קשה שקוטעת תשובה באמצע ומשאירה תשובה חלקית ומטעה.

התשובה: ניקוז מבוקר: מה שהתחיל מסיים, שום דבר חדש לא נכנס.

הכלים שמאחורי החלק

- **Envoy Proxy + WASM (מוכן מהמדף):** Envoy הוא שומר בכניסה לרשת, ו-WASM מאפשר להכניס לו כלל בדיקה קטן משלנו בלי לשנות אותו. בדיקת התחום רצה כמסנן בתוך ה-proxy, לפני שהבקשה בכלל מגיעה לקוד האפליקציה.
- **Istio Service Mesh (מוכן מהמדף):** רשת שמציבה שומר קטן ליד כל שירות, כך שגם שיחות בין שירותים פנימיים עוברות דרך בקרה. היא מבטיחה שגם בקשה שבאה משירות פנימי באשכול מכבדת את מצב העצירה.
- **Circuit Breaker (מוכן מהמדף):** מפסק חשמל חכם שבמקום לכבות הכול בבת אחת, קודם מוריד את העומס ורק אחר כך מנתק. כשיש יותר מדי אירועים חשודים, התעבורה מוגבלת חלקית כאזהרה מוקדמת, ורק במקרה קיצון מגיעה עצירה מלאה.
- **מצב בזיכרון שמתעדכן מ-Kafka (הרכבנו):** במקום לשאול מרחוק על כל בקשה, מחזיקים לוח קטן ליד השער שמתעדכן ברקע, כמו לוח משמרות תלוי על הקיר. כל שער שומר בזיכרון את מצב כל תחום, ובדיקת בקשה היא חיפוש אחד מהיר שלא נוגע ברשת.
- **ניקוז מבוקר (החלטה שלנו):** כשסוגרים ברז לא קוטעים את הכוס שכבר מתמלאת, אלא נותנים לה לסיים ורק אז סוגרים. בקשות שכבר התחילו ממשיכות עד מגבלת זמן של כ-10 שניות, אבל אף בקשה חדשה לא נכנסת.

נתונים

GatewayState: status (ACTIVE או SUSPENDED), since (מאז מתי), reason (למה), last_command_id (הפקודה שגרמה לכך)

קלט ופלט: בקשת HTTP עם פרטי תחום ← העברה למודל או 503 עם הסבר

חלק 4: ועדה ולא אדם אחד

פקודה תקפה דורשת שלושה מתוך חמישה חותמים. מפתח אחד שנגנב לא מספיק.

איך זה עובד

- חמישה חברי ועדה עצמאיים, וכל אחד מחזיק חלק ממפתח.
- שלושה מהם מפיקים יחד חתימה אחת (FROST).
- השער בודק חתימה אחת, כרגיל. המורכבות קורית לפני השליחה.
- מפתח חשוד נחסם מיד ברשימת ביטולים. החלפת מפתחות נעשית בחפיפה של 24 שעות.

מה זה מגן מפניו

האיום: מפתח אחד נגנב, או שאדם אחד בוועדה מושחת.

התשובה: חתימה אחת לא מספיקה, והמפתח החשוד נחסם מיד.

הכלים שמאחורי החלק

- **FROST (חתימת סף) (מוכן מהמדף):** חתימה משותפת שבה מספיקים M מתוך N אנשים כדי ליצור חתימה אחת, כמו כספת שנפתחת בשלושה מפתחות מחמישה. שלושה מחמשת חברי ועדת הבטיחות חותמים יחד, והשער רואה חתימה אחת רגילה ובודק אותה בקלות.

- **Vault Transit (מוכן מהמדף):** שירות בתוך Vault שחותם ומצפין בשבילך, כך שהמפתח לא יוצא לעולם, כמו נוטריון שמחזיק את החותמת. בקשות חתימה והחלפת מפתחות עוברות דרכו, וגרסאות ישנות נשמרות כדי לאמת פקודות מהעבר.
- **CRL / OCSP (מוכן מהמדף):** רשימה שחורה של תעודות שבוטלו, ממש כמו חסימת כרטיס אשראי שאבד, שנבדקת בכל שימוש. מפתח שנחשד כפגום נכנס לרשימת הביטולים, והשער בודק אותה בכל אימות ודוחה אותו מיד.
- **יומן שקוף בהשראת Certificate Transparency (החלטה שלנו):** יומן ציבורי שלא ניתן לשנות, שבו כל שינוי נרשם לעיני כולם, בדומה ללוח מודעות שאי אפשר למחוק ממנו. כל הוספה, הסרה או החלפה של מפתח מוסמך נרשמת ביומן פומבי, וכל אחד יכול לראות מי מחזיק סמכות עצירה.
- **ועדה במקום מפתח יחיד (החלטה שלנו):** במקום אדם אחד עם הכפתור, קבוצה צריכה להסכים, כמו שני מפתחות בשיגור טילים. פקודה נחשבת מאושרת רק כשמספר חברי ועדה עצמאיים חתמו עליה, וכך אין יחיד שיכול לעצור או לחדש לבד.

נתונים

AuthorizedSigner: key_id (מזהה המפתח), public_key_share (חלק המפתח של החבר), valid_from / valid_until (תוקף המפתח), role (תפקיד, למשל רגולטור)

קלט ופלט: חתימות חלקיות משלושה חברים ← חתימה אחת, ועדכון ביומן הציבורי

חלק 5: עצירה ממוקדת

לא תמיד צריך לעצור הכול. אפשר רק מודל אחד, אזור אחד או יכולת אחת.

איך זה עובד

- כל בקשה מסומנת: מודל, לקוח, אזור ויכולות.
- הפקודה כוללת תחום בתחביר היררכי, למשל region:eu-*.
- מנוע חוקים בודק אם הבקשה נופלת בתוך תחום פעיל.
- אם המידע חסר או פגום, חוסמים רק את הבקשה הזו ורושמים.

איזו טעות זה מונע

האיום: עצירה רחבה מדי בטעות, או מידע חסר שמבלבל את השער.

התשובה: עובדים ב-Fail-Closed ברמת הבקשה בלבד, בלי לפגוע בכל השאר.

הכלים שמאחורי החלק

- **OPA + Rego (מוכן מהמדף):** OPA הוא מנוע שמחזיק את חוקי ההרשאות במקום נפרד מהקוד. Rego היא השפה שבה כותבים לו את החוקים. אפשר לכתוב בה למשל "כל אירופה" בשורה אחת. תחומים כמו region:eu-* נכתבים כחוקים, והשער שואל את המנוע, לכל בקשה, אם היא נופלת בתוך עצירה פעילה. זה הלב של מנוע התאמת התחומים: הרשימה הפעילה של מה שנעצר.
- **בהשראת LaunchDarkly (מוכן מהמדף):** LaunchDarkly הוא כלי שמדליק ומכבה תכונות במוצר לפי כללים על הבקשה, למשל "רק למשתמשים מאירופה". אנחנו לא משתמשים בו. לקחנו ממנו את הרעיון ואת הארכיטקטורה. מנוע התאמת התחומים הוא החלק ששואל, לכל בקשה שנכנסת, "האם היא נופלת בתוך תחום שנעצר עכשיו?". אצלו הכללים נשמרים מקומית בזיכרון, ולכן ההכרעה לוקחת שבריר מילישנייה, בלי לפנות לרשת.
- **Bloom Filter (מוכן מהמדף):** בודק מהיר ששואל אם משהו כנראה בקבוצה, אפשר לטעות לחיוב אבל אף פעם לא לשלילה, כמו שומר שמכיר פנים. כשיש אלפי חוקי עצירה, הוא מדלג מיד על בקשות שבוודאות לא קשורות, ורק חשודות עוברות ל-OPA.

- **ברירת מחדל: Fail-Closed לבקשה בודדת (החלטה שלנו):** Fail-Closed הוא מונח מהנדסת בטיחות: כשמערכת לא בטוחה במשהו, היא נוטה למצב הבטוח, שבו הדבר החשוד לא עובר. כמו שומר בכניסה שלא מכניס אורח בלי תעודה, אבל משאיר את הבניין עצמו פתוח לכל מי שיש לו תעודה. באנגלית זה המונח המקובל, ולכן השארנו אותו כך. בקשה שבה חסר מידע על התחום שלה, או שהמידע פגום, נחסמת ומתועדת. הבקשה הזו בלבד. כל שאר הבקשות ממשיכות כרגיל.

נתונים

ScopeRule: pattern (model:gpt-x/capability:code-execution), status (SUSPENDED), created_by (מי יצר), created_at (מתי)

קלט ופלט: הקשר הבקשה וחוקים פעילים ← חסימה או מעבר

חלק 6: כשהתשתית נופלת

נפל Redis? נפל Kafka? לכל תחום הוגדר מראש מה עושים: להמשיך או לחסום.

איך זה עובד

- לכל תחום מוגדרת מדיניות: עובדים ב-Fail-Open או ב-Fail-Closed.
- תחום מסוכן (הרצת קוד, למשל) עובד ב-Fail-Closed. תחום כללי עובד ב-Fail-Open עם התרעה.
- כל שער שומר על הדיסק את המצב האחרון שהכיר.
- אין עדכון טרי מעל חמש דקות? עוברים אוטומטית למדיניות הבטוחה.

מה זה מגן מפניו

האיום: מפילים את מרכז הבקרה כדי "לעוור" את המנגנון.

התשובה: שתיקה נחשבת סימן אזהרה, והשער עובר למצב הבטוח (Dead Man's Switch).

הכלים שמאחורי החלק

- **Chaos Monkey / Gremlin (מוכן מהמדף):** כלים שמפילים בכוונה חלקים במערכת כדי לראות אם היא מחזיקה, כמו תרגיל כיבוי אש לפני שיש שריפה. מנתקים את Redis ומעכבים את Kafka בסביבת בדיקה כדי לוודא שמצב Fail-Safe באמת פועל.
- **SQLite (מוכן מהמדף):** מסד נתונים קטן שנשמר בקובץ אחד על המחשב עצמו, בלי שרת, כמו פנקס כיס. לכל שער יש עותק מקומי של המצב האחרון, כדי שיזכור אותו גם אחרי הפעלה מחדש וגם בלי קשר לתשתית המרכזית.
- **Dead Man's Switch (מוכן מהמדף):** מנגנון שדורש אות חיים קבוע, ואם האות נפסק, מניחים שמשהו רע קרה, כמו נהג רכבת שחייב ללחוץ על כפתור. מרכז הבקרה שולח פעימות, ואם הן נעלמות, השער עובר מעצמו למצב הבטוח שהוגדר מראש.
- **מדיניות נפרדת לכל תחום (החלטה שלנו):** לא כל דבר דורש את אותה רמת זהירות, כמו שבמעבדה הדלת לחומרים מסוכנים נעולה יותר מדלת המשרד. כל תחום מקבל החלטה משלו: תעבורה מסוכנת כמו הרצת קוד עובדת ב-Fail-Closed, ותעבורה רגילה עובדת ב-Fail-Open עם התרעה.

נתונים

LocalStateSnapshot: scope_id (התחום), status (ACTIVE או SUSPENDED), last_updated (עדכון אחרון), fail_safe_policy (FAIL_OPEN או FAIL_CLOSED)

קלט ופלט: פעימות חיים ומדיניות לכל תחום ← מצב אכיפה גם בלי קשר לתשתית

חלק 7: שמירה על השומר

המנגנון מנטר גם את עצמו: מי חתם, כמה ומתי, ומתריע כשמשהו חורג.

איך זה עובד

- לכל מפתח נשמר דפוס רגיל: כמה פקודות בשעה, באילו שעות, על אילו תחומים.
- כל פקודה נבדקת מול הדפוס.
- חריגה של יותר מ-3 סטיות תקן מקפיצה התרעה.
- ההתרעה זורמת למערכת האבטחה שהצוות כבר רואה כל יום.

מה זה מגן מפניו

האיום: מפתח תקף נפל לידיים הלא נכונות ושולח פקודות בשעות מוזרות.

התשובה: הדפוס חורג, וההתרעה נשלחת.

הכלים שמאחורי החלק

- **Splunk / Elastic Security (מוכן מהמדף):** מערכות שמרכזות אירועי אבטחה ממקומות רבים למסך אחד, כמו חדר בקרה של עירייה. כל שימוש בכפתור העצירה מוזרם אליהן, וצוות האבטחה רואה את ההתרעות על המסך שהוא כבר מכיר.
- **UEBA (מוכן מהמדף):** ניתוח התנהגות שמכיר מה רגיל לכל ישות ומתריע כשהיא סוטה, כמו בנק שמבחין בקנייה לא אופיינית בכרטיס שלך. לכל מפתח חתימה יש פרופיל של תדירות, שעות ותחומים רגילים, ושימוש חריג מקפיץ התרעה.
- **Z-score דטרמיניסטי, בלי AI (החלטה שלנו):** מספר שמודד כמה הערך רחוק מהממוצע, ואותה קלט תמיד נותן אותה תוצאה, כמו סרגל ולא כמו נחש. אם מספר הפקודות בשעה האחרונה חורג בלמעלה מ-3 סטיות תקן מהרגיל של המפתח, נשלחת התרעה.

נתונים

SignerBaseline: key_id (המפתח), commands_per_hour_avg / stddev (ממוצע וסטיית תקן), typical_active_hours (שעות פעילות רגילות), typical_scopes (תחומים רגילים)

קלט ופלט: יומן הפקודות של כל מפתח ← ציון חריגה, והתרעה

חלק 8: כמה אזורים

הפקודה צריכה להגיע לכל אזור שבו המודל רץ, גם כשהקשר בין אזורים נקטע.

איך זה עובד

- בכל אזור אשכול Kafka וצי שערים משלו.
- הפקודות משוכפלות בין האזורים באותו סדר.
- אם הקשר נקטע, כל אזור ממשיך לפי הפקודה האחרונה שקיבל.
- כשהקשר חוזר, הכול מסתנכרן.

מה זה מגן מפניו

האיום: ניתוק בין אזורים, או נפילה של דאטה-סנטר שלם.

התשובה: כל אזור ממשיך לפעול, ומסתנכרן בחזרה אחר כך.

הכלים שמאחורי החלק

- **Cluster Linking / MirrorMaker 2 (מוכן מהמדף):** כלים שמעתיקים אוטומטית ערוץ הודעות מאשכול Kafka אחד לאחר, כמו מכונת צילום שעובדת כל הזמן. פקודת עצירה מהמרכז הראשי מועתקת לאשכול של כל אזור, וכל אזור פועל לפי האחרונה שקיבל גם אם הקשר נקטע.
- **Anycast (מוכן מהמדף):** אותה כתובת רשת משודרת מכמה מקומות בעולם, והאינטרנט שולח כל אחד לקרוב והזמין, כמו מספר חירום אחד לכל המדינה. מרכז הבקרה זמין תחת כתובת אחת מכל מקום, וגם אם דאטה סנטר שלם נופל הבקשות עוברות לאזור אחר.
- **מדד עיכוב גלוי לניטור (הרכבנו):** מספר שמראה כמה זמן אזור מאחר אחרי המרכז, כמו מד שמראה כמה שעון אחד מפגר אחרי האחר. לכל אזור נשמר העיכוב של ההעתקה ב-Kafka, ומסך הניטור מציג אותו.

נתונים

RegionClusterStatus: region_id (האזור), kafka_replication_lag_ms (כמה מילישניות מפגר), last_synced_command_id (הפקודה האחרונה שסונכרנה)

קלט ופלט: פקודות מהמרכז ← מצב מסונכרן בכל אזור

חלק 9: ספריות למפתחים

חיבור המנגנון למערכת קיימת בכעשר שורות קוד.

איך זה עובד

- מפרט API אחד (OpenAPI) הוא מקור האמת.
- ממנו נוצרות ספריות אוטומטית לכל שפה.
- בין השער למרכז הבקרה: gRPC, מהיר ובעל חוזה טיפוסים.
- תוכנת ביניים מוכנה לשרתי FastAPI ו-Express.

איזו טעות זה מונע

האיום: אימוץ שדורש לשכתב קוד קיים, ולכן אף אחד לא מאמץ.

התשובה: מתחברים בכעשר שורות, בלי לגעת בקוד הקיים.

הכלים שמאחורי החלק

- **OpenAPI Generator (מוכן מהמדף):** כלי שקורא תיאור אחד של ה-API ומייצר מזה ספריות מוכנות בשפות רבות, כמו מדפסת שמוציאה את אותו ספר בכל שפה. ממפרט OpenAPI יחיד נוצרות ספריות לקוח ביותר מ-40 שפות, כך שאפשר להשתלב בכ-10 שורות קוד.
- **gRPC + Protobuf (מוכן מהמדף):** דרך מהירה לשירותים לדבר זה עם זה בפורמט בינארי קומפקטי וחוזה ברור, כמו מברק מקודד במקום מכתב. התקשורת הפנימית בין השער למרכז הבקרה רצה עליהם, מעל HTTP/2.
- **FastAPI (מוכן מהמדף):** ספרייה פופולרית בפייתון לבניית שרתי אינטרנט, שאפשר לחבר אליה רכיב ביניים שרץ על כל בקשה. אנחנו מספקים לה רכיב ביניים מוכן שבודק מול השער על כל בקשה נכנסת, בלי לגעת בקוד הקיים.
- **Express (מוכן מהמדף):** ספרייה פופולרית ב-Node.js לבניית שרתי אינטרנט, שתומכת ברכיבי ביניים שחוצים כל בקשה. אנחנו מספקים לה רכיב ביניים מוכן שבודק מול השער על כל בקשה, בלי לשנות את הנתיבים הקיימים.

נתונים

OpenAPISpec + ProtobufSchema: OpenAPISpec (מקור אמת יחיד ל-API), ProtobufSchema (חוזה טיפוסים לתקשורת פנימית)

קלט ופלט: מפרט אחד ← ספריות לכל שפה, ותוכנת ביניים

חלק 10: הדמיה לפני עצירה

לפני שחותמים על עצירה אמיתית, רואים בדיוק מה ייפגע.

איך זה עובד

- פעולה שלישית, DRY_RUN, לצד STOP ו-RESUME.
- השער מריץ את אותה התאמה מול תעבורה אמיתית.
- אבל לא חוסם כלום.
- מחזיר דוח: כמה בקשות, מאילו לקוחות ואילו אזורים.

איזו טעות זה מונע

האיום: region:eu-* שתופס בטעות משהו שלא התכוונו אליו.

התשובה: רואים את רדיוס הפגיעה המדויק לפני החתימה.

הכלים שמאחורי החלק

- **Traffic Mirroring (מוכן מהמדף):** העתקת תעבורה אמיתית למקום נפרד לצורכי בדיקה, בלי שהמשתמש ירגיש, כמו חזרה גנרלית עם קהל אמיתי. במצב סימולציה מריצים את מנוע ההתאמה על תעבורה חיה כדי לראות מה היה נחסם, בלי לחסום בפועל.
- **פעולת DRY_RUN (החלטה שלנו):** הרצה לניסיון שמראה מה היה קורה בלי לעשות את זה באמת, כמו מדידת בגד בלי לקנות. סוג פקודה שלישי לצד STOP ו-RESUME, שמחזיר דוח כמה בקשות, מאילו לקוחות ואזורים היו נחסמות.

נתונים

commands.action: action (STOP / RESUME / DRY_RUN)

קלט ופלט: תחום מוצע ← דוח השפעה, בלי חסימה

חלק 11: תפוגה אוטומטית

עצירה שנשכחה לא נשארת לנצח: אחרי ארבע שעות היא חוזרת מעצמה, אלא אם חותמים מחדש.

איך זה עובד

- לכל פקודת עצירה יש expires_at, למשל 4 שעות.
- אין חידוש? המצב חוזר ל-ACTIVE.
- חידוש דורש פקודה חדשה וחתומה.
- כל חידוש, או היעדרו, נשאר ביומן הביקורת.

איזו טעות זה מונע

האיום: עצירה שנשכחה לצמיתות כי מישהו התכוון לחדש ושכח.

התשובה: התפוגה אוטומטית, והחידוש משאיר עקבות.

הכלים שמאחורי החלק

- **Temporal.io (מוכן מהמדף):** פלטפורמה שזוכרת תהליכים ארוכים גם אם השרת כבה באמצע, כמו מזכירה שרושמת הכול ביומן ולא בראש. טיימר התפוגה של כל עצירה מנוהל כתהליך עמיד, ובסיומו המצב חוזר ל-ACTIVE.
- **תפוגה כברירת מחדל (החלטה שלנו):** כל עצירה מגיעה עם שעון חול, ובסיומו היא נגמרת מעצמה, כמו חניה בכחול לבן שנגמרת בלי לבקש. לכל פקודת STOP יש זמן תפוגה, למשל 4 שעות, וחידוש דורש פקודה חדשה וחתומה שנרשמת ביומן.

נתונים

scope_states.expires_at: (מתי העצירה פגה) expires_at

קלט ופלט: פקודת עצירה עם תוקף ← חזרה אוטומטית ל-ACTIVE

ההחלטות

Ed25519 או RSA?

בחרנו Ed25519. האימות מהיר בהרבה והחתימה קטנה, 64 בייט. כשכל אלפית שנייה בשער חשובה, זה מכריע.

RSA נשאר בחוץ.

אדם אחד או ועדה?

בחרנו ועדה, 3 מתוך 5. מפתח אחד שנחשף נותן שליטה מלאה, ואדם מושחת אחד לא נעצר. ועדה פותרת את "מי שומר על השומרים".

המחיר: תיאום בין החותמים.

להמשיך או לחסום כשהכול נופל?

בחרנו: תלוי בתחום. תחום מסוכן עובד ב-Fail-Closed, תחום כללי עובד ב-Fail-Open עם התרעה. מנגנון בטיחות לא אמור להפיל שירות תקין.

המחיר: מדיניות לכל תחום.

AI או סטטיסטיקה?

בחרנו סטטיסטיקה פשוטה. רכיב אבטחה קריטי חייב להיות דטרמיניסטי, שקוף וניתן להסבר. Z-score מול דפוס רגיל עושה את העבודה.

המחיר: פחות "חכם".

לקטוע או לנקז?

בחרנו לנקז. בקשות שכבר התחילו מסיימות עד 10 שניות. תשובה חלקית ומטעה מסוכנת יותר מהמתנה קצרה.

המחיר: עצירה לא מיידית לגמרי.

לעצור לתמיד או לזמן?

בחרנו עצירה עם תפוגה. השבתות שנשכחות הן כשל תפעולי מוכר. תפוגה אוטומטית, וחידוש בחתימה חדשה, משאירים עקבות.

המחיר: צריך לזכור לחדש.

מה השתנה מהגרסה הראשונה

קודם	עכשיו
מפתח אחד	ועדה, 3 מתוך 5
מצב בזיכרון של תהליך אחד	Redis ו-Kafka, בכמה אזורים
עצירה כללית	עצירה ממוקדת לפי תחום
לא התמודדנו עם נפילה	מדיניות לכל תחום, ו-Dead Man's Switch
יומן עם שרשור גיבובים	יומן ציבורי שקוף, וניטור חריגות